CSE 2006 - Programming in Java

Course Type: LP Credits: 3

Prepared by
Mr. C. S Rajpoot
School of Computing Science and Engineering
VIT Bhopal University

Unit-1 Contents

- **Java Introduction**
- Java Hello World, Java JVM, JRE and JDK, Difference between C & C++, Java Variables, Java Data Types, Java Operators, Java Input and Output, Java Expressions & Blocks, Java Comment
- **Java Flow Control**
- Java if...else, Java switch Statement, Java for Loop, Java for-each Loop, Java while Loop, Java break Statement, Java continue Statement

Difference between C & C++

C	C++
C was developed by Dennis Ritchie between the year 1969 and 1973 at AT&T Bell Labs.	C++ was developed by Bjarne Stroustrup in 1979.
C does no support polymorphism, encapsulation, and inheritance which means that C does not support object oriented programming.	C++ supports polymorphism, encapsulation, and inheritance because it is an object oriented programming language.
C is a subset of C++.	C++ is a superset of C.
C contains 32 keywords.	C++ contains 63 keywords.
For the development of code, C supports procedural programming.	C++ is known as hybrid language because C++ supports both procedural and object oriented programming paradigms.
Data and functions are separated in C because it is a procedural programming language.	Data and functions are encapsulated together in form of an object in C++.

Difference between C & C++

C	C++
C does not support information hiding.	Data is hidden by the Encapsulation to ensure that data structures and operators are used as intended.
Built-in data types is supported in C.	Built-in & user-defined data types is supported in C++.
C is a function driven language because C is a procedural programming language.	C++ is an object driven language because it is an object oriented programming.
Function and operator overloading is not supported in C.	Function and operator overloading is supported by C++.
C is a function-driven language.	C++ is an object-driven language
Functions in C are not defined inside structures.	Functions can be used inside a structure in C++.
Namespace features are not present inside the C.	Namespace is used by C++, which avoid name collisions.

Difference between C & C++

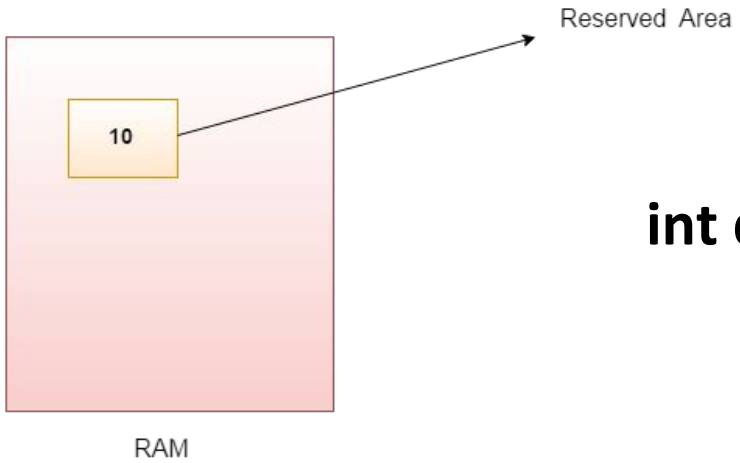
C	C++
Header file used by C is stdio.h.	Header file used by C++ is iostream.h.
Reference variables are not supported by C.	Reference variables are supported by C++.
Virtual and friend functions are not supported by C.	Virtual and friend functions are supported by C++.
C does not support inheritance.	C++ supports inheritance.
Instead of focusing on data, C focuses on method or process.	C++ focuses on data instead of focusing on method or procedure.
C provides malloc() and calloc() functions for dynamic memory allocation, and free() for memory de-allocation.	C++ provides new operator for memory allocation and delete operator for memory de-allocation.
Direct support for exception handling is not supported by C.	Exception handling is supported by C++.

Difference between C & C++

C	C++
scanf() and printf() functions are used for input/output in C.	cin and cout are used for input/output in C++.
C structures don't have access modifiers.	C ++ structures have access modifiers.
C follows the top-down approach	C++ follows the Bottom-up approach
There is no strict type checking in C programming language.	Strict type checking in done in C++. So many programs that run well in C compiler will result in many warnings and errors under C++ compiler.
C does not support overloading	C++ does support overloading

Java Variables

- A variable is a container which holds the value while the Java program is executed. ***A variable is assigned with a data type.***
- Variable is a name of memory location. There are three types of variables in java: **local, instance and static.**
- There are two types of data types in Java: **primitive and non-primitive.**
- A variable is the name of a reserved area **allocated in memory.**
- In other words, it is a name of the memory location. It is a combination of "vary + able" which means its value can be changed.



int data=50; //Here data is variable

Types of Variables

- **There are three types of variables in Java:**
- **1) Local Variable**
- A variable declared inside the body of the method is called local variable. You can use this variable only within that method and the other methods in the class aren't even aware that the variable exists.
- A local variable cannot be defined with "static" keyword.
- **2) Instance Variable**
- A variable declared inside the class but outside the body of the method, is called an instance variable. It is not declared as static.
- It is called an instance variable because its value is instance-specific and is not shared among instances.
- **3) Static variable**
- A variable that is declared as static is called a static variable. It cannot be local. You can create a single copy of the static variable and share it among all the instances of the class. Memory allocation for static variables happens only once when the class is loaded in the memory.

Types of Variables

- **Example to understand the types of variables in java**

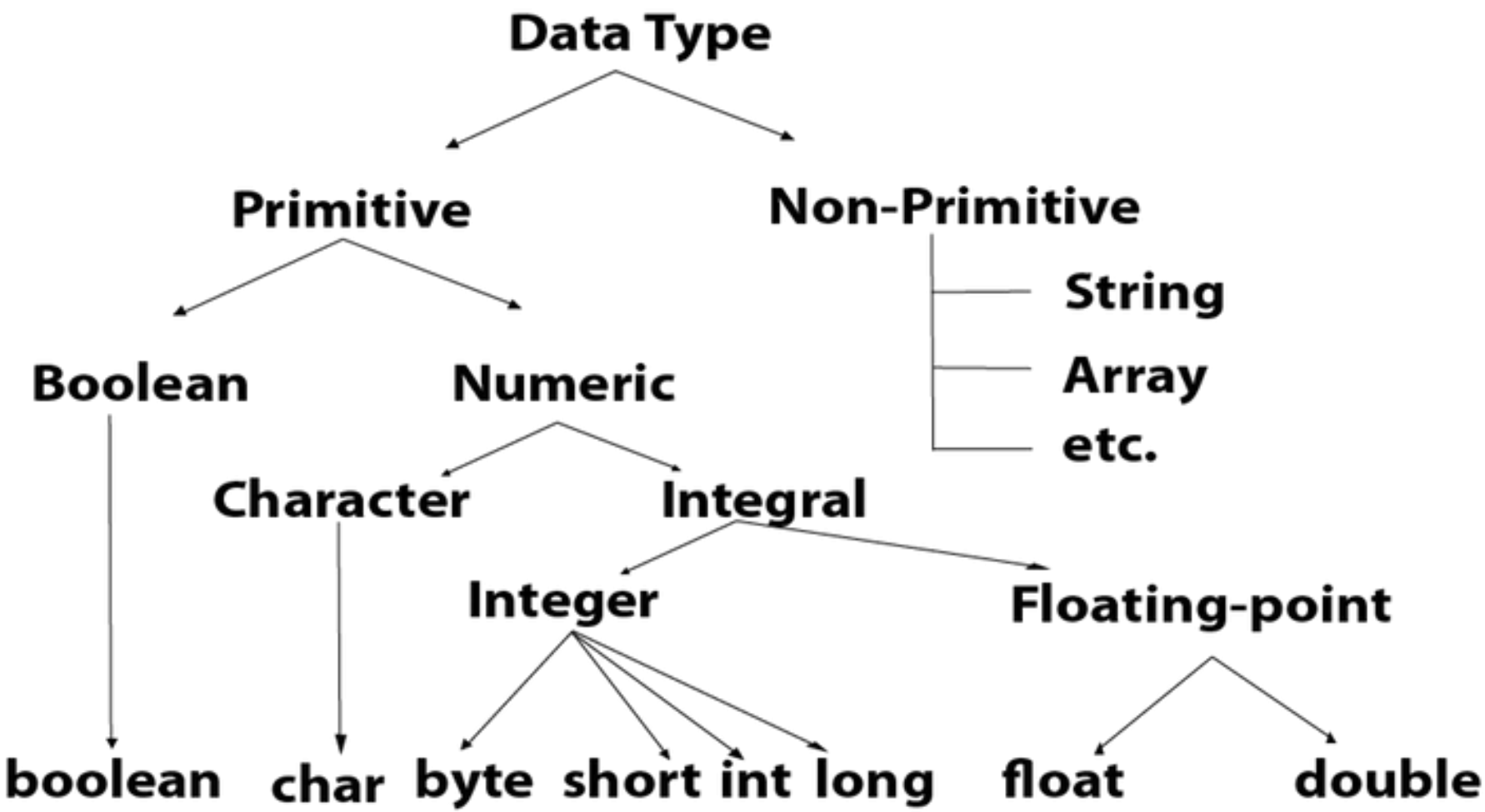
```
public class A
{
   static int m=100; //static variable
   void method()
   {
      int n=90; //local variable
   }
   public static void main(String args[])
   {
      int data=50; //instance variable
   }
}//end of class
```

Java Data Types

- Data types specify the different sizes and values that can be stored in the variable. There are two types of data types in Java:
- **Primitive data types:** The primitive data types include **boolean, char, byte, short, int, long, float and double.**
- **Non-primitive data types:** The non-primitive data types include **Classes, Interfaces, and Arrays.**
- Java Primitive Data Types
- In Java language, primitive data types are the building blocks of data manipulation.
- These are the most basic data types available in Java language.

- Java is a statically-typed programming language.
- It means, all variables must be declared before its use. That is why we need to declare variable's type and name.

Java Data Types



Java Data Types

Data Type	Default Value	Default size
boolean	false	1 bit
char	'\u0000'	2 byte
byte	0	1 byte
short	0	2 byte
int	0	4 byte
long	0L	8 byte
float	0.0f	4 byte
double	0.0d	8 byte

Java Data Types

Boolean Data Type

- The Boolean data type is used to store only two possible values: **true and false.** This data type is used for simple flags that track true/false conditions.
- The Boolean data type specifies one bit of information, but its "size" can't be defined precisely.
- **Example:** Boolean one = **false**

Byte Data Type

- The byte data type is an example of primitive data type. It isan 8-bit signed two's complement integer. Its value-range lies between -128 to 127 (inclusive). Its minimum value is -128 and maximum value is 127. Its default value is 0.
- The byte data type is used to save memory in large arrays where the memory savings is most required. It saves space because a byte is 4 times smaller than an integer. It can also be used in place of "int" data type.
- **Example:** **byte** a = 10, **byte** b = -20

Java Data Types

Short Data Type

- The short data type is a 16-bit signed two's complement integer. Its value-range lies between -32,768 to 32,767 (inclusive). Its minimum value is -32,768 and maximum value is 32,767. Its default value is 0.
- The short data type can also be used to save memory just like byte data type. A short data type is 2 times smaller than an integer.
- **short** s = 10000, **short** r = -5000

Int Data Type

- The int data type is a 32-bit signed two's complement integer. Its value-range lies between - 2,147,483,648 (-2^31) to 2,147,483,647 (2^31 -1) (inclusive). Its minimum value is - 2,147,483,648and maximum value is 2,147,483,647. Its default value is 0.
- The int data type is generally used as a default data type for integral values unless if there is no problem about memory.
- **Example:**
- **int** a = 100000, **int** b = -200000

Java Data Types

Long Data Type

- The long data type is a 64-bit two's complement integer. Its value-range lies between -9,223,372,036,854,775,808(-2^63) to 9,223,372,036,854,775,807(2^63 -1)(inclusive). Its minimum value is -9,223,372,036,854,775,808and maximum value is 9,223,372,036,854,775,807. Its default value is 0. The long data type is used when you need a range of values more than those provided by int.
- **Example: long** a = 100000L, **long** b = -200000L

Float Data Type

- The float data type is a single-precision 32-bit IEEE 754 floating point.Its value range is unlimited. It is recommended to use a float (instead of double) if you need to save memory in large arrays of floating point numbers. The float data type should never be used for precise values, such as currency. Its default value is 0.0F.
- **Example: float** f1 = 234.5f

Java Data Types

Double Data Type

- The double data type is a double-precision 64-bit IEEE 754 floating point. Its value range is unlimited. The double data type is generally used for decimal values just like float. The double data type also should never be used for precise values, such as currency. Its default value is 0.0d.
- **Example:**
- **double** d1 = 12.3

Char Data Type

- The char data type is a single 16-bit Unicode character. Its value-range lies between '\u0000' (or 0) to '\uffff' (or 65,535 inclusive).The char data type is used to store characters.
- **Example:**
- **char** letterA = 'A'

Java Operators

- Operators are symbols that perform operations on variables and values. For example, + is an operator used for addition, while * is also an operator used for multiplication.
- **Operators in Java can be classified into 5 types:**

1. Arithmetic Operators
2. Assignment Operators
3. Relational Operators
4. Logical Operators
5. Unary Operators
6. Bitwise Operators

Java Operators

- **1. Java Arithmetic Operators**
- Arithmetic operators are used to perform arithmetic operations on variables and data.

Operator	Name	Description	Example
+	Addition	Adds together two values	x + y
-	Subtraction	Subtracts one value from another	x - y
*	Multiplication	Multiplies two values	x * y
/	Division	Divides one value by another	x / y
%	Modulus	Returns the division remainder	x % y
++	Increment	Increases the value of a variable by 1	++x
--	Decrement	Decreases the value of a variable by 1	--x

https://www.w3schools.com/java/java_operators.asp

Java Operators

- **Java Logical Operators**
- Logical operators are used to determine the logic between variables or values:

Operator	Name	Description	Example
&&	Logical and	Returns true if both statements are true	x < 5 && x < 10
||	Logical or	Returns true if one of the statements is true	x < 5 || x < 4
!	Logical not	Reverse the result, returns false if the result is true	!(x < 5 && x < 10)

Java Operators

```
// Java Arithmetic operations
import java.io.*;
class arith
{    public static void main(String args[])
     {try{
       DataInputStream din=new DataInputStream(System.in);
       System.out.println("Enter the 2-no's");
       int i=Integer.parseInt(din.readLine());
       int j=Integer.parseInt(din.readLine());
       System.out.println("addition is" + (i+j));
       System.out.println("subtraction is" + (i-j));
       System.out.println("multiplication is" + (i*j));
       System.out.println("divisionion is" + (i/j));
       System.out.println("modulo division  is" +(i%j));
            }catch(Exception e) {}
     }
}
```

Java Operators

- **Java Operator Precedence**

Operator Type	Category	Precedence
Unary	postfix	*expr*++ *expr*--
	prefix	++*expr* --*expr* +*expr* -*expr* ~ !
Arithmetic	multiplicative	* / %
	additive	+ -
Shift	shift	<< >> >>>
Relational	comparison	< > <= >= instanceof
	equality	== !=
Bitwise	bitwise AND	&
	bitwise exclusive OR	^
	bitwise inclusive OR	|
Logical	logical AND	&&
	logical OR	||
Ternary	ternary	? :
Assignment	assignment	= += -= *= /= %= &= ^= |= <<= >>= >>>=

Java Operators

- https://www.javatpoint.com/operators-in-java

Java Operators

- **Java Unary Operator**
- The Java unary operators require only one operand.

Unary operators are used to perform various operations i.e.:

- incrementing/decrementing a value by one
- negating an expression
- inverting the value of a boolean

Java Operators

- **Java Unary Operator Example: ++ and --**

```
public class OperatorExample{
public static void main(String args[]){
int x=10;
System.out.println(x++);//10 (11)
System.out.println(++x);//12
System.out.println(x--);//12 (11)
System.out.println(--x);//10
}}
```

Output:

10
12
12
10

Java Operators

- **Java Unary Operator Example 2: ++ and --**

```
public class OperatorExample{
public static void main(String args[]){
int a=10;
int b=10;
System.out.println(a++ + ++a);//10+12=22
System.out.println(b++ + b++);//10+11=21

}}
```

Output:
22
21

Java Operators

- **Java Unary Operator Example: ~ and !**

```
public class OperatorExample{
public static void main(String args[]){
int a=10;
int b=-10;
boolean c=true;
boolean d=false;
System.out.println(~a);
//-11 (minus of total positive value which starts from 0)
System.out.println(~b);
//9 (positive of total minus, positive starts from 0)
System.out.println(!c);   //false (opposite of boolean value)
System.out.println(!d);  //true
}}
```

Output:

```
-11
9
false
true
```

Java Operators

- **Java Arithmetic Operator Example: Expression**

```
public class OperatorExample{
public static void main(String args[]){
System.out.println(10*10/5+3-1*4/2);
}}
```

Output:
21

Java Operators

- **Java Left Shift Operator**
- The Java left shift operator << is used to shift all of the bits in a value to the left side of a specified number of times.

Java Left Shift Operator Example

```
public class OperatorExample{
public static void main(String args[]){
System.out.println(10<<2);//10*2^2=10*4=40
System.out.println(10<<3);//10*2^3=10*8=80
System.out.println(20<<2);//20*2^2=20*4=80
System.out.println(15<<4);//15*2^4=15*16=240
}}
```

Output:

```
40
80
80
240
```

Java Operators

- **Java Right Shift Operator**
- The Java right shift operator >> is used to move the value of the left operand to right by the number of bits specified by the right operand.

Java Right Shift Operator Example

```
public OperatorExample{
public static void main(String args[]){
System.out.println(10>>2);   //10/2^2=10/4=2
System.out.println(20>>2);   //20/2^2=20/4=5
System.out.println(20>>3);   //20/2^3=20/8=2
}}
```

Output:

2
5
2

Java Operators

- **Java Assignment Operator**
- Java assignment operator is one of the most common operators. It is used to assign the value on its right to the operand on its left.

Java Assignment Operator Example

```
public class OperatorExample{
public static void main(String args[]){
int a=10;
int b=20;
a+=4;//a=a+4 (a=10+4)
b-=4;//b=b-4 (b=20-4)
System.out.println(a);
System.out.println(b);
}}
```

Output:
14
16

Java Operators

- **Java Assignment Operator Example**

```
public class OperatorExample{
public static void main(String[] args){
int a=10;
a+=3;//10+3
System.out.println(a);
a-=4;//13-4
System.out.println(a);
a*=2;//9*2
System.out.println(a);
a/=2;//18/2
System.out.println(a);
}}
```

Output:

13
9
18
9

Java Operators

- **Java Assignment Operator Example: Adding short**

```
public class OperatorExample{
public static void main(String args[]){
short a=10;
short b=10;
//a+=b;//a=a+b internally so fine
a=a+b;//Compile time error because 10+10=20 now int
System.out.println(a);
}}
```

Output:
Compile time error

Java Operators

- **After type cast:**

```
public class OperatorExample{
public static void main(String args[]){
short a=10;
short b=10;
a=(short)(a+b);//20 which is int now converted to short
System.out.println(a);
}}
```

Output:
20

Java Expressions

- **What is statement in Java?**
- In Java, a **statement** is an executable instruction that tells the compiler what to perform. It forms a complete command to be executed and can include one or more **expressions**. A sentence forms a complete idea that can include one or more clauses.

- **Expression Statements**
- Expression is an essential building block of any Java program
- . Generally, it is used to generate a new value. Sometimes, we can also assign a value to a variable
- . In Java, expression is the combination of values, variables, operators, and method calls.

Java Expressions

- **There are three types of expressions in Java:**
- Expressions that **produce** a value. For example, **(6+9), (9%2), (pi*radius) + 2**. Note that the expression enclosed in the parentheses will be evaluate first, after that rest of the expression.
- Expressions that **assign** a value. For example, **number = 90, pi = 3.14.**
- Expression that **neither produces any result nor assigns a value**. For example, **increment** or **decrement** a value by using increment or decrement operator respectively, **method invocation**, etc. These expressions modify the value of a variable or state (memory) of a program. For example, **count++, int sum = a + b**; The expression changes only the value of the variable **sum**. The value of variables **a** and **b** do not change, so it is also a side effect.

Flow of Control

- Java executes one statement after the other in the order they are written
- Many Java statements are flow control statements:

Alternation: if, if else, switch

Looping: for, while, do while

Escapes: break, continue, return

If – The Conditional Statement

- The if statement evaluates an expression and if that evaluation is true then the specified action is taken

```
if ( x < 10 ) x = 10;
```

- If the value of x is less than 10, make x equal to 10
- It could have been written:

```
if ( x < 10 )
x = 10;
```

- Or, alternatively:

```
if ( x < 10 ) { x = 10; }
```

If… else Statement

- The if … else statement evaluates an expression and performs one action if that evaluation is true or a different action if it is false.

```
if (x != oldx) {
  System.out.print("x was changed");
}
else {
  System.out.print("x is unchanged");
}
```

Nested if … else

```
if ( myVal > 100 ) {
  if ( remainderOn == true) {
     myVal = mVal % 100;
  }
  else {
   myVal = myVal / 100.0;
  }
}
else
{
  System.out.print("myVal is in range");
}
```

else if

- Useful for choosing between alternatives:

```
if ( n == 1 ) {
  // execute code block #1
}
else if ( j == 2 ) {
  // execute code block #2
}
else {
  // if all previous tests have failed, execute
  code block #3
}
```

The switch Statement

```
switch ( n ) {
  case 1:
   // execute code block #1
   break;
  case 2:
   // execute code block #2
   break;
   default:
   // if all previous tests fail then
   //execute code block #4
   break;
}
```

The for loop

- Loop n times

```
for ( i = 0; i < n; i++ ) {
  // this code body will execute n times
  // ifrom  0 to n-1
}
```

- Nested for:

```
for ( j = 0; j < 10; j++ ) {
  for ( i = 0; i < 20; i++ ){
   // this code body will execute 200 times
  }
}
```

while loops

```
while(response == 1) {
  System.out.print( “ID =” + userID[n]);
  n++;
  response = readInt( “Enter “);
}
```

do {… } while loops

```
do {
  System.out.print( “ID =” + userID[n] );
  n++;
  response = readInt( “Enter ” );
}while (response == 1);
```

Break

- A break statement causes an exit from the innermost containing while, do, for or switch statement.

```
for ( int i = 0; i < maxID, i++ ) {
  if ( userID[i] == targetID ) {
   index = i;
   break;
  }
} // program jumps here after break
```

Continue

- Can only be used with while, do or for.
- The continue statement causes the innermost loop to start the next iteration immediately

```
for ( int i = 0; i < maxID; i++ ) {
  if ( userID[i] != -1 ) continue;
  System.out.print( "UserID " + i + " :" +
  userID);
}
```

Arrays

- An array is a collection of similar items
- An array has a fixed:
  - name
  - type
  - length
- These must be declared when the array is created.
- Arrays sizes cannot be changed during the execution of the code

myArray =

3	6	3	1	6	3	4	1
0	1	2	3	4	5	6	7

myArray has room for 8 elements

- the elements are accessed by their index
- in Java, array indices start at 0

Declaring Arrays

int myArray[];

declares *myArray* to be an array of integers

myArray = **new** int[8];

sets up 8 integer-sized spaces in memory, labelled *myArray[0]* to *myArray[7]*

int myArray[] = **new** int[8];

combines the two statements in one line

Assigning Values

- refer to the array elements by index to store values in them.

```
myArray[0] = 3;
myArray[1] = 6;
myArray[2] = 3;   ...
```

- can create and initialise in one step:

```
int myArray[] = {3, 6, 3, 1, 6, 3, 4, 1};
```

Iterating Through Arrays

- *for* loops are useful when dealing with arrays:

```
for (int i = 0; i < myArray.length; i++) {
  myArray[i] = getsomevalue();
}
```

Multi Dimensional Array

Two dimensional array:

```
data_type[][] array_name = new data_type[x][y];
```

```
For example:
int[][] arr = new int[10][20]; or
Int arr[][]= new int[10][20];
```

Three dimensional array:

```
int[][][] threeD_arr = new int[10][20][30];
```

Declaring the Array

1. Declare the array

```
private Student studentList[];
```

– this declares studentList

2 .Create the array

```
studentList = new Student[10];
```

– this sets up 10 spaces in memory that can hold references to Student objects

3. Create Student objects and add them to the array:

```
studentList[0] = new Student("Cathy", "Computing");
```